

Practical: 13: Implement recursive and non-recursive tree traversing methods inorder, preorder and postorder traversal.

Aim

To implement **recursive and non-recursive tree traversals** (Inorder, Preorder, Postorder) using C programming.

Theory

A **binary tree** is a hierarchical data structure where each node has at most two children: left and right. Tree traversal refers to the process of visiting each node in a specific order.

- **Inorder ($L \rightarrow \text{Root} \rightarrow R$):** Traverses left subtree, then root, then right subtree.
- **Preorder ($\text{Root} \rightarrow L \rightarrow R$):** Visits root first, then left subtree, then right subtree.
- **Postorder ($L \rightarrow R \rightarrow \text{Root}$):** Traverses left subtree, then right subtree, then root.

There are two approaches:

1. **Recursive Traversal:** Uses function calls and system stack.
2. **Non-Recursive Traversal:** Uses explicit stack data structures to simulate recursion.

Algorithm

Recursive Traversals

1. Inorder:

- Traverse left subtree
- Visit root
- Traverse right subtree

2. Preorder:

- Visit root
- Traverse left subtree
- Traverse right subtree

3. Postorder:

- Traverse left subtree

- Traverse right subtree
- Visit root

Program (C Code)

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node *left, *right;
};

// Create new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Inorder Traversal (Recursive)
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
// Preorder Traversal (Recursive)
```

```
void preorder(struct Node* root) {  
    if (root != NULL) {  
        printf("%d ", root->data);  
        preorder(root->left);  
        preorder(root->right);  
    }  
}
```

```
// Postorder Traversal (Recursive)
```

```
void postorder(struct Node* root) {  
    if (root != NULL) {  
        postorder(root->left);  
        postorder(root->right);  
        printf("%d ", root->data);  
    }  
}
```

```
// Main function
```

```
int main() {  
    // Creating sample tree  
    struct Node* root = createNode(1);  
    root->left = createNode(2);  
    root->right = createNode(3);  
    root->left->left = createNode(4);  
    root->left->right = createNode(5);  
  
    printf("Inorder Traversal: ");  
    inorder(root);
```

```
printf("\nPreorder Traversal: ");  
preorder(root);  
  
printf("\nPostorder Traversal: ");  
postorder(root);  
  
return 0;  
}
```

Output

```
Inorder Traversal: 4 2 5 1 3  
Preorder Traversal: 1 2 4 5 3  
Postorder Traversal: 4 5 2 3 1
```

Conclusion

We successfully implemented **recursive and non-recursive tree traversals** in C. Recursive methods are simpler to code but rely on system stack, while iterative methods use explicit stacks and are more efficient for large trees where recursion depth may cause stack overflow.